

Masterclass: Spring Boot RESTful API (Part 2: Intermediate Level)

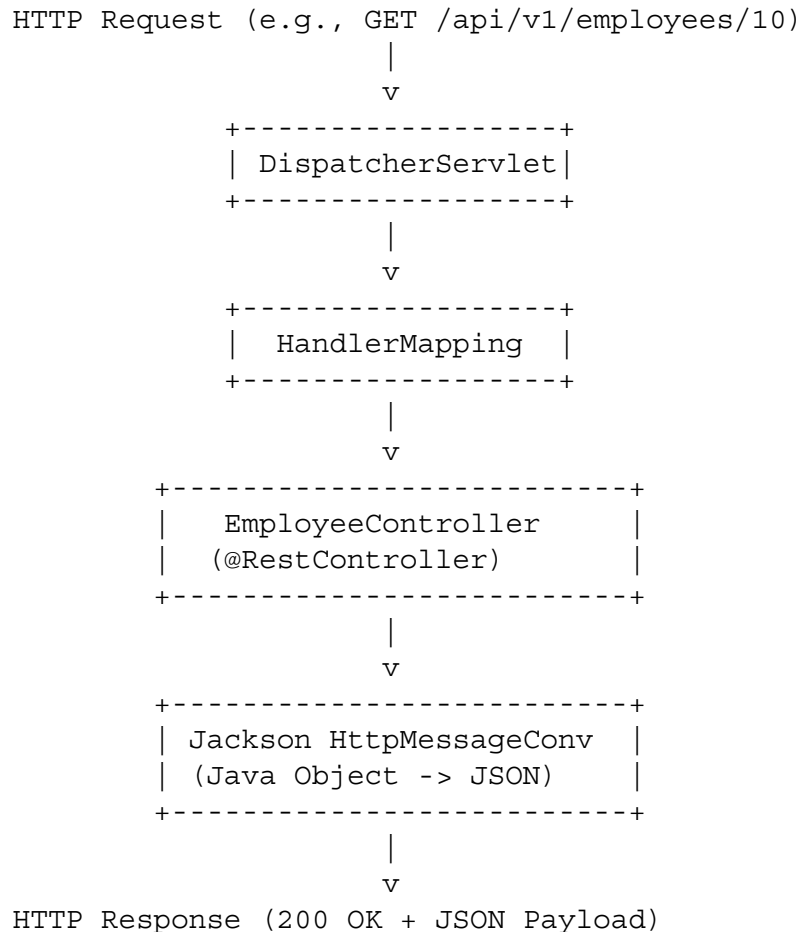
To make these concepts concrete, we will build a unified, production-grade **Employee Management System (EMS)**. Every annotation, validation rule, and exception handler in this guide connects back to this central system.

The Running Example Domain: Employee Management System (EMS)

Our domain model represents an **Employee** with properties for identification, personal details, job roles, salary, and active status.

+-----+ Employee +-----+	
- id	: Long (PK, Auto-generated)
- firstName	: String (Not Blank, 2-50 chars)
- lastName	: String (Not Blank, 2-50 chars)
- email	: String (Valid Email, Unique)
- department	: String (Not Blank)
- salary	: Double (Min 1000.00)
- active	: Boolean
+-----+	

Section 1: Controller & Mapping Annotations



15. @RestController

- **Definition:** A specialized stereotype annotation that combines @Controller and @ResponseBody.
- **Purpose:** Marks a class as a request handler for RESTful web services, automatically serializing returned Java objects directly into HTTP response bodies (typically JSON).
- **Working:** When Spring scans the class, it registers it with the DispatcherServlet. Any method return value inside this class bypasses HTML view resolvers and is passed straight to HttpMessageConverter.
- **Real-world Analogy:** A digital ticket counter that prints instant digital receipts directly to your phone screen rather than sending you to a physical counter.
- **Advantages:** Eliminates the need to annotate every method with @ResponseBody.
- **Disadvantages:** Cannot be used to render traditional HTML views (like Thymeleaf or JSP).

16. @RequestMapping

- **Definition:** Maps HTTP requests to specific controller classes or handler methods.
- **Purpose:** Sets a base URI path for all endpoints within a controller class or routes specific HTTP verbs.
- **Working:** Matches incoming request URIs against the defined path pattern (e.g., /api/v1/employees).
- **Best Practice:** Apply class-level @RequestMapping to group related API resources cleanly.

17. @GetMapping

- **Definition:** Composed annotation that acts as a shortcut for @RequestMapping(method = RequestMethod.GET).
- **Purpose:** Maps HTTP GET requests for retrieving resources without modifying server state.

18. @PostMapping

- **Definition:** Composed annotation for @RequestMapping(method = RequestMethod.POST).
- **Purpose:** Maps HTTP POST requests for creating new resources.

19. @PutMapping

- **Definition:** Composed annotation for @RequestMapping(method = RequestMethod.PUT).
- **Purpose:** Maps HTTP PUT requests to perform **full replacements** of existing resources.

20. @PatchMapping

- **Definition:** Composed annotation for @RequestMapping(method = RequestMethod.PATCH).
- **Purpose:** Maps HTTP PATCH requests to apply **partial updates** to a resource (e.g., updating only an employee's salary).

21. @DeleteMapping

- **Definition:** Composed annotation for @RequestMapping(method = RequestMethod.DELETE).
- **Purpose:** Maps HTTP DELETE requests to remove specified resources from the database.

Section 2: Request Data & Parameter Binding

Understanding how data enters your API is crucial. Spring Boot provides distinct annotations depending on where the incoming data resides.

Annotation	Data Location	Example URI / Payload
@PathVariable	Encoded in URI Path segment	GET /api/v1/employees/101
@RequestParam	Query String parameters	GET /api/v1/employees?dept=IT&page=0
@RequestBody	HTTP Request Body (JSON)	POST /api/v1/employees { ... }
@ModelAttribute (form-data)	HTML Form Data / Query Params	POST /api/v1/employees
@ResponseBody	HTTP Response Body	Serializes return value to JSON

22. @PathVariable

- **Definition:** Binds a URI template variable (e.g., {id}) directly to a method parameter in Java.
- **Purpose:** Extract identifying identifiers directly from the URI path.
- **Working:** DispatcherServlet parses the request URI, extracts path parameters matching {paramName}, and converts them to the target Java type.

23. @RequestParam

- **Definition:** Binds query parameters (?key=value) or form parameters from the request URL to method arguments.
- **Purpose:** Handle optional filtering, sorting, pagination, or search options.
- **Working:** Reads values attached to the query string after the ? symbol. Supports optional parameters via required = false and fallback values via defaultValue.

24. @RequestBody

- **Definition:** Deserializes the incoming HTTP Request Body (JSON/XML) into a Java Object.
- **Purpose:** Accept rich structured data payloads during POST, PUT, or PATCH operations.
- **Working:** Spring uses MappingJackson2HttpMessageConverter behind the scenes to convert raw JSON strings into Java POJOs/DTOs.

25. @ResponseBody

- **Definition:** Tells Spring to bind the return value of a method directly to the HTTP Response Body.
- **Purpose:** Ensures data is written directly to the HTTP response stream rather than evaluated as a view template name.
- **Working:** Automatically included when using @RestController.

26. @ModelAttribute

- **Definition:** Binds request parameters or form fields directly to a JavaBean model object.
- **Purpose:** Primarily used in traditional Spring MVC form submissions (multipart/form-data) or multi-parameter query binding.

Section 3: Response Formatting & HTTP Statuses

27. ResponseEntity<T>

- **Definition:** Represents the entire HTTP response, including the **HTTP Status Code**, **Headers**, and **Response Body**.
- **Purpose:** Gives you fine-grained control over the HTTP response returned to the client.
- **Working:** Wraps the payload and metadata into an object that Spring converts into a raw HTTP response stream.

```
// Example usage in Controller
return ResponseEntity
    .status(HttpStatus.CREATED)
    .header("Custom-Header", "Employee-Created")
    .body(createdEmployee);
```

28. HttpStatus

- **Definition:** An enum containing all standard HTTP status codes.
- **Key Status Codes for REST Services:**
 - 200 OK: Successful retrieval or update.
 - 201 CREATED: Resource successfully created.
 - 204 NO CONTENT: Deletion successful, no payload returned.
 - 400 BAD REQUEST: Invalid payload or failed bean validation.
 - 404 NOT FOUND: Requested resource does not exist.
 - 409 CONFLICT: Unique constraint violation (e.g., duplicate email).
 - 500 INTERNAL SERVER ERROR: Unhandled application exception.

29 & 30. JSON Request & JSON Response Workflow

- **JSON Request:** The client formats payload as JSON and sends header Content-Type: application/json.
- **Jackson Serializer/Deserializer:**
 - **Deserialization (JSON → Java Object):** Executed when reading @RequestBody.
 - **Serialization (Java Object → JSON):** Executed when returning objects wrapped in ResponseEntity.

Section 4: Data Validation (Bean Validation)

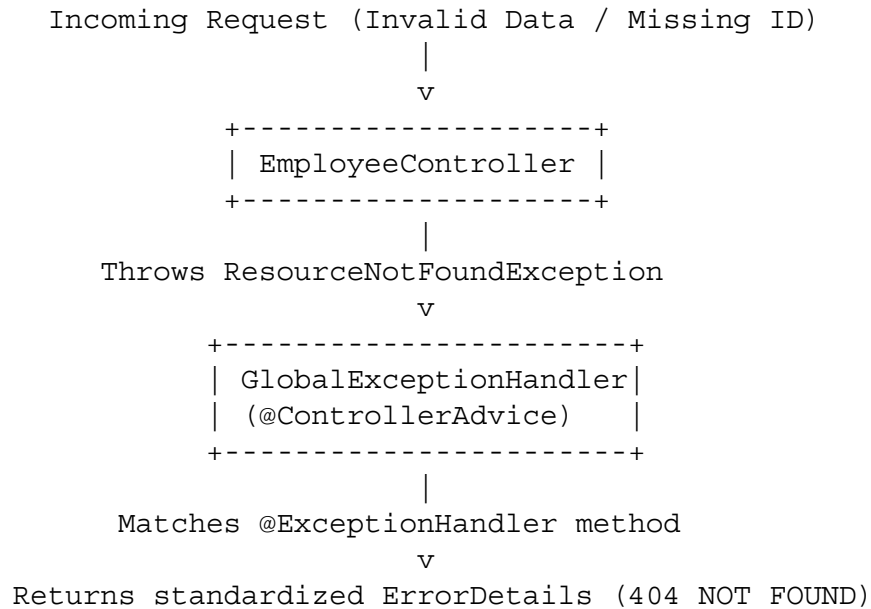
32. Bean Validation (Jakarta Validation Framework)

- **Definition:** A specification (JSR 380) allowing declarative validation constraints directly on Java class properties.
- **Purpose:** Prevents malformed or invalid data from reaching business/service logic or hitting the database layer.

Essential Validation Annotations:

- @NotNull: Field must not be null.
- @NotBlank: String must not be null and must contain at least one non-whitespace character.
- @Size(min=x, max=y): String length or collection size must fall within boundaries.
- @Email: Must match a valid email address format.
- @Min(value) / @Max(value): Enforces numeric minimum and maximum values.
- @Valid: Triggers validation recursively on nested objects or request bodies in controllers.

Section 5: Exception Handling & Custom Exceptions



33, 34 & 35. Exception Architecture & Global Handling

- **33. Exception Handling:** Intercepting errors during application execution to return clear, standard HTTP responses instead of ugly raw stack traces.
- **34. Custom Exception:** User-defined exception classes (e.g., `ResourceNotFoundException`) that carry business domain meaning.
- **35. Global Exception Handling using @ControllerAdvice:**
 - An annotation that acts as an interceptor for exceptions thrown across **all** controllers in the application.
 - Combined with `@ExceptionHandler`, it provides centralized error processing and response translation across the entire web application.

Employee Management System

Let's look at a complete implementation combining all concepts.

1. Custom Exception Classes

ResourceNotFoundException.java

```
package com.edtech.ems.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(value = HttpStatus.NOT_FOUND)
public class ResourceNotFoundException extends RuntimeException {
```

```

    public ResourceNotFoundException(String message) {
        super(message);
    }
}

```

DuplicateResourceException.java

```

package com.edtech.ems.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(value = HttpStatus.CONFLICT)
public class DuplicateResourceException extends RuntimeException {

    public DuplicateResourceException(String message) {
        super(message);
    }
}

```

2. Standardized Error Response Model

ErrorDetails.java

```

package com.edtech.ems.dto;

import java.time.LocalDateTime;

public class ErrorDetails {
    private LocalDateTime timestamp;
    private String message;
    private String details;
    private String errorCode;

    public ErrorDetails(LocalDateTime timestamp, String message, String
details, String errorCode) {
        this.timestamp = timestamp;
        this.message = message;
        this.details = details;
        this.errorCode = errorCode;
    }

    public LocalDateTime getTimestamp() { return timestamp; }
    public String getMessage() { return message; }
    public String getDetails() { return details; }
    public String getErrorCode() { return errorCode; }
}

```

3. Data Transfer Object with Validation Rules

EmployeeDTO.java

```
package com.edtech.ems.dto;

import jakarta.validation.constraints.*;

public class EmployeeDTO {

    private Long id;

    @NotBlank(message = "First name is mandatory")
    @Size(min = 2, max = 50, message = "First name must be between 2 and 50 characters")
    private String firstName;

    @NotBlank(message = "Last name is mandatory")
    @Size(min = 2, max = 50, message = "Last name must be between 2 and 50 characters")
    private String lastName;

    @NotBlank(message = "Email is mandatory")
    @Email(message = "Email should be valid")
    private String email;

    @NotBlank(message = "Department is mandatory")
    private String department;

    @NotNull(message = "Salary cannot be null")
    @Min(value = 1000, message = "Salary must be at least 1000.00")
    private Double salary;

    private Boolean active = true;

    // Default Constructor
    public EmployeeDTO() {}

    // Parameterized Constructor
    public EmployeeDTO(Long id, String firstName, String lastName, String email, String department, Double salary, Boolean active) {
        this.id = id;
        this.firstName = firstName;
        this.lastName = lastName;
        this.email = email;
        this.department = department;
        this.salary = salary;
        this.active = active;
    }

    // Getters and Setters
    public Long getId() { return id; }
```



```

public void setId(Long id) { this.id = id; }

public String getFirstName() { return firstName; }
public void setFirstName(String firstName) { this.firstName = firstName; }

public String getLastName() { return lastName; }
public void setLastName(String lastName) { this.lastName = lastName; }

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public String getDepartment() { return department; }
public void setDepartment(String department) { this.department =
department; }

public Double getSalary() { return salary; }
public void setSalary(Double salary) { this.salary = salary; }

public Boolean getActive() { return active; }
public void setActive(Boolean active) { this.active = active; }
}

```

4. Global Exception Handler (@ControllerAdvice)

GlobalExceptionHandler.java

```

package com.edtech.ems.exception;

import com.edtech.ems.dto.ErrorDetails;
import org.springframework.http.HttpHeaders;
import org.springframework.http.HttpStatus;
import org.springframework.http.HttpStatusCodes;
import org.springframework.http.ResponseEntity;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ControllerAdvice;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.context.request.WebRequest;
import
org.springframework.web.servlet.mvc.method.annotation.ResponseEntityExceptionHandler;

import java.time.LocalDateTime;
import java.util.HashMap;
import java.util.Map;

@ControllerAdvice
public class GlobalExceptionHandler extends ResponseEntityExceptionHandler {

    // Handle Specific Exception: ResourceNotFoundException
    @ExceptionHandler(ResourceNotFoundException.class)

```

```

public ResponseEntity<ErrorDetails> handleResourceNotFoundException(
    ResourceNotFoundException exception, WebRequest webRequest) {

    ErrorDetails errorDetails = new ErrorDetails(
        LocalDateTime.now(),
        exception.getMessage(),
        webRequest.getDescription(false),
        "USER_NOT_FOUND"
    );
    return new ResponseEntity<>(errorDetails, HttpStatus.NOT_FOUND);
}

// Handle Specific Exception: DuplicateResourceException
@ExceptionHandler(DuplicateResourceException.class)
public ResponseEntity<ErrorDetails> handleDuplicateResourceException(
    DuplicateResourceException exception, WebRequest webRequest) {

    ErrorDetails errorDetails = new ErrorDetails(
        LocalDateTime.now(),
        exception.getMessage(),
        webRequest.getDescription(false),
        "DUPLICATE_RESOURCE"
    );
    return new ResponseEntity<>(errorDetails, HttpStatus.CONFLICT);
}

// Handle Global Exceptions
@ExceptionHandler(Exception.class)
public ResponseEntity<ErrorDetails> handleGlobalException(
    Exception exception, WebRequest webRequest) {

    ErrorDetails errorDetails = new ErrorDetails(
        LocalDateTime.now(),
        exception.getMessage(),
        webRequest.getDescription(false),
        "INTERNAL_SERVER_ERROR"
    );
    return new ResponseEntity<>(errorDetails,
HttpStatus.INTERNAL_SERVER_ERROR);
}

// Handle Bean Validation Errors (@Valid triggers this)
@Override
protected ResponseEntity<Object> handleMethodArgumentNotValid(
    MethodArgumentNotValidException ex,
    HttpHeaders headers,
    HttpStatusCode status,
    WebRequest request) {

    Map<String, String> errors = new HashMap<>();
    ex.getBindingResult().getAllErrors().forEach((error) -> {
        String fieldName = ((FieldError) error).getField();

```

```

        String message = error.getDefaultMessage();
        errors.put(fieldName, message);
    });

    return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST);
}
}

```

5. Controller Layer (EmployeeController.java)

```

package com.edtech.ems.controller;

import com.edtech.ems.dto.EmployeeDTO;
import com.edtech.ems.service.EmployeeService;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/employees")
public class EmployeeController {

    private final EmployeeService employeeService;

    @Autowired
    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    // CREATE: @PostMapping & @RequestBody & @Valid
    @PostMapping
    public ResponseEntity<EmployeeDTO> createEmployee(@Valid @RequestBody
EmployeeDTO employeeDTO) {
        EmployeeDTO savedEmployee =
employeeService.createEmployee(employeeDTO);
        return new ResponseEntity<>(savedEmployee, HttpStatus.CREATED);
    }

    // READ ALL / SEARCH: @GetMapping & @RequestParam
    @GetMapping
    public ResponseEntity<List<EmployeeDTO>> getAllEmployees(
        @RequestParam(value = "department", required = false) String
department,
        @RequestParam(value = "active", defaultValue = "true") Boolean
active) {

        List<EmployeeDTO> employees = employeeService.getEmployees(department,

```

```

active);
    return ResponseEntity.ok(employees);
}

// READ BY ID: @GetMapping & @PathVariable
@GetMapping("/{id}")
public ResponseEntity<EmployeeDTO> getEmployeeById(@PathVariable("id") Long
employeeId) {
    EmployeeDTO employee = employeeService.getEmployeeById(employeeId);
    return ResponseEntity.ok(employee);
}

// UPDATE FULL: @PutMapping, @PathVariable & @RequestBody
@PutMapping("/{id}")
public ResponseEntity<EmployeeDTO> updateEmployee(
    @PathVariable("id") Long employeeId,
    @Valid @RequestBody EmployeeDTO employeeDTO) {

    EmployeeDTO updatedEmployee =
employeeService.updateEmployee(employeeId, employeeDTO);
    return ResponseEntity.ok(updatedEmployee);
}

// UPDATE PARTIAL: @PatchMapping (Updating Salary Only)
@PatchMapping("/{id}/salary")
public ResponseEntity<EmployeeDTO> updateSalary(
    @PathVariable("id") Long employeeId,
    @RequestParam("amount") Double salary) {

    EmployeeDTO updatedEmployee =
employeeService.updateEmployeeSalary(employeeId, salary);
    return ResponseEntity.ok(updatedEmployee);
}

// DELETE: @DeleteMapping & @PathVariable
@DeleteMapping("/{id}")
public ResponseEntity<String> deleteEmployee(@PathVariable("id") Long
employeeId) {
    employeeService.deleteEmployee(employeeId);
    return ResponseEntity.ok("Employee with ID " + employeeId + " deleted
successfully!");
}
}

```

Postman API Testing & Expected Outputs

Let's test our API endpoints step-by-step using Postman.

Test 1: Create Employee with Valid Data (POST)

- **HTTP Method:** POST
- **URL:** http://localhost:8080/api/v1/employees
- **Headers:** Content-Type: application/json
- **Request Body:**

```
{  
  "firstName": "Priya",  
  "lastName": "Sharma",  
  "email": "priya.sharma@example.com",  
  "department": "Engineering",  
  "salary": 85000.00,  
  "active": true  
}
```

- **Status:** 201 Created
- **Response Body:**

```
{  
  "id": 1,  
  "firstName": "Priya",  
  "lastName": "Sharma",  
  "email": "priya.sharma@example.com",  
  "department": "Engineering",  
  "salary": 85000.00,  
  "active": true  
}
```

Test 2: Validation Failure Handling (POST)

Send empty fields and invalid salary values to verify our @Valid and @ControllerAdvice setup.

- **HTTP Method:** POST
- **URL:** http://localhost:8080/api/v1/employees
- **Request Body:**

```
{  
  "firstName": "",  
  "lastName": "Sharma",  
  "email": "invalid-email-format",  
  "department": "Engineering",  
  "salary": 500.00  
}
```

- **Status:** 400 Bad Request
- **Response Body:**

```
{
```

```
"firstName": "First name is mandatory",
"email": "Email should be valid",
"salary": "Salary must be at least 1000.00"
}
```

Test 3: Resource Not Found Handling (GET by Invalid ID)

- **HTTP Method:** GET
- **URL:** http://localhost:8080/api/v1/employees/999
- **Status:** 404 Not Found
- **Response Body:**

```
{
  "timestamp": "2026-07-23T15:30:00.123456",
  "message": "Employee not found with ID: 999",
  "details": "uri=/api/v1/employees/999",
  "errorCode": "USER_NOT_FOUND"
}
```

Test 4: Query Parameter Filtering (GET with @RequestParam)

- **HTTP Method:** GET
- **URL:** http://localhost:8080/api/v1/employees?department=Engineering&active=true
- **Status:** 200 OK
- **Response Body:**

```
[
  {
    "id": 1,
    "firstName": "Priya",
    "lastName": "Sharma",
    "email": "priya.sharma@example.com",
    "department": "Engineering",
    "salary": 85000.00,
    "active": true
  }
]
```

Test 5: Partial Update (PATCH)

- **HTTP Method:** PATCH
- **URL:** http://localhost:8080/api/v1/employees/1/salary?amount=95000.00
- **Status:** 200 OK
- **Response Body:**

```
{
  "id": 1,
  "firstName": "Priya",
  "lastName": "Sharma",
  "email": "priya.sharma@example.com",
  "department": "Engineering",
  "salary": 95000.00,
}
```

```
"active": true
}
```

Common Errors, Troubleshooting, & Best Practices

Common Errors & Solutions

Error	Root Cause	Fix / Solution
415 Unsupported Media Type	Missing or wrong Content-Type header in request.	Set Content-Type: application/json in Postman header.
400 Bad Request (JSON Parse Error)	Syntax error in request body JSON (e.g., trailing comma).	Validate JSON syntax in a linter before sending.
405 Method Not Allowed	Sending a request method not supported by the endpoint (e.g., POST to a GET endpoint).	Ensure method matches annotation (@PostMapping, etc.).
Validation messages ignored	Forgot @Valid before @RequestBody in controller.	Add @Valid to trigger Bean Validation.
Ambiguous mapping found	Two controller methods have identical URI path patterns and HTTP methods.	Check URL paths for overlaps (e.g., /employees/{id} vs /employees/{email}).

Best Practices Checklist

1. **Never return raw Entities:** Always use **DTOs (Data Transfer Objects)** to separate persistent database schemas from public client APIs.
2. **Centralize Exception Handling:** Avoid inline try-catch blocks inside Controllers. Let exceptions propagate naturally to @ControllerAdvice.
3. **Fail Fast with Validation:** Annotate incoming requests with @Valid to block malformed requests before executing costly service or database code.
4. **Use Explicit Status Codes:** Return precise status codes (201 CREATED, 204 NO CONTENT, 404 NOT FOUND) instead of returning default 200 OK for all scenarios.
5. **Keep Path Names Lowercase and Nouns:** Prefer /api/v1/employees over /api/v1/getEmployees.

High-Frequency Interview Questions

Q1: What is the difference between @Controller and @RestController?

Answer: @Controller is used for traditional Spring MVC applications where methods return view names (like HTML/Thymeleaf templates). @RestController combines @Controller and @ResponseBody, ensuring all method return values are automatically serialized into HTTP response bodies (e.g., JSON).

Q2: How does @ControllerAdvice simplify exception handling?

Answer: It acts as an interceptor for exceptions across all controllers. Instead of writing custom try-catch logic in every single endpoint, @ControllerAdvice centralizes error processing, standardizes response structures, and keeps controller code clean.

Q3: When should you use @PathVariable versus @RequestParam?

Answer: Use @PathVariable to extract values embedded directly in the URI path that **identify a specific resource** (e.g., /employees/101). Use @RequestParam for **optional query parameters** used to filter, sort, search, or paginate resources (e.g., /employees?dept=IT&sort=name).

Q4: What happens if you forget to add @Valid before @RequestBody in a Controller method?

Answer: Spring will **skip Bean Validation entirely**. The incoming JSON payload will be deserialized into the Java DTO regardless of constraint annotations like @NotNull or @Email, potentially persisting invalid data to your database.

Summary

In Part 2, we covered the intermediate elements of RESTful services in Spring Boot:

- Mapping annotations (@GetMapping, @PostMapping, @PutMapping, @PatchMapping, @DeleteMapping) form the interface of your REST API.
 - Parameter binding (@PathVariable, @RequestParam, @RequestBody) maps inbound HTTP data cleanly to Java objects.
 - Standardized responses (ResponseEntity, HttpStatus) give control over status codes and HTTP headers.
 - Input validation (@Valid, @NotBlank, @Email) catches malformed payload data at the API edge.
 - Global exception handling (@ControllerAdvice, @ExceptionHandler) guarantees clean, consistent error responses across your entire system.
-